SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART I

ALGORITHMIC DESIGN THROUGH PSEUDOCODE

Assignment Details	Information
Course	Design and Analysis of Algorithms
Course Outcome	CO4 – Solve optimization problems using Dynamic Programming and Greedy strategies
Nature of Assignment	Individual
Duration	One Week
Total Marks	50
Submission Date	04-11-26
Student Name	Suvedhan G
Register Number	192524381

TASK 1 – ALGORITHM DESIGN REVIEW

Diagnose and Repair Poor Pseudocode (6 Marks)

Task 1.1 – Diagnose the Design (2 Marks)

No.	**Deficiency**	**Explanation**
1	Inputs are not defined	It does not specify package weights, values or vehicle capacity.
2	Sorting criterion is missing	It says arrange packages but does not specify the sorting order.
3	Meaning of “best” is unclear	It does not explain whether the best package has maximum value or maximum value-to-weight ratio.

4	Feasibility is not checked	It does not check whether a package fits within the remaining capacity.
5	Fractional selection is missing	It does not explain how to handle a package that cannot completely fit.
6	Remaining capacity is not updated	The available vehicle capacity after loading is not maintained.
7	Termination condition is incomplete	It does not specify when to stop if all packages are processed.
8	Result construction is missing	It does not maintain the selected package portions and accumulated value.

Task 1.2 – Repair the Design (3 Marks)

Greedy rule: Select packages in descending order of value-to-weight ratio. If a package does not completely fit, take only the fraction that fits.

```
ALGORITHM FractionalKnapsack(Packages, Capacity)
INPUT:
   Packages containing weight and value of each package
   Capacity – maximum vehicle capacity
OUTPUT:
   Selected package portions and maximum total value
BEGIN
   TotalValue ← 0
   RemainingCapacity ← Capacity
   FOR each package P DO
      P.Ratio ← P.Value / P.Weight
   END FOR
   SORT Packages in descending order of Ratio
   FOR each package P in sorted order DO
      IF RemainingCapacity = 0 THEN
         BREAK
      END IF
      IF P.Weight ≤ RemainingCapacity THEN
```

```
        SelectedWeight ← P.Weight
        SelectedValue ← P.Value
      ELSE
        SelectedWeight ← RemainingCapacity
        SelectedValue ← P.Value *
                    (RemainingCapacity / P.Weight)
      END IF
      TotalValue ← TotalValue + SelectedValue
      RemainingCapacity ← RemainingCapacity - SelectedWeight
      RECORD P.ID and SelectedWeight
    END FOR
    RETURN Selected packages and TotalValue
END
```

Task 1.3 – Design Defence (1 Mark)

The rewritten solution qualifies as pseudocode because it describes the algorithm using language-independent statements such as SORT, IF, FOR and RETURN. It clearly identifies inputs, outputs, selection criteria, conditions and updates. It does not depend on any particular programming language, compiler or programming syntax. Another engineer can understand and implement the algorithm using any suitable programming language.

TASK 2 – FROM BUSINESS REQUIREMENT TO GREEDY PSEUDOCODE

Job Sequencing with Deadlines (8 Marks)

Task 2.1 – Model the Requirement (2 Marks)

Component	**Description**
Input	Job ID, deadline and profit of each job.
Output	Final job schedule and maximum total profit.
Objective	Maximise total profit from completed jobs.
Constraints	Each job requires one slot. Only one job can occupy a slot. Each job must finish before or on its deadline.
Decision	Whether to select a job and which available slot to assign.
Required information	Job deadline, profit, available time slots and current schedule.

Task 2.2 – Formulate a Greedy Hypothesis (1 Mark)

Greedy-choice rule: Select jobs in descending order of profit and assign each job to the latest available time slot on or before its deadline.

Justification: Selecting higher-profit jobs first helps maximise profit, while placing each job in the latest feasible slot preserves earlier slots for jobs with smaller deadlines.

Task 2.3 – Design the Algorithm (3 Marks)

```
ALGORITHM JobSequencing(Jobs)
INPUT:
   Jobs containing ID, Deadline and Profit
OUTPUT:
   Final schedule and Maximum Profit
BEGIN
   MaximumDeadline ← maximum deadline among all jobs
   CREATE Schedule[1 ... MaximumDeadline]
   INITIALISE all slots as EMPTY
   TotalProfit ← 0
   SORT Jobs in descending order of Profit
   FOR each Job J in sorted order DO
      SlotFound ← FALSE
      FOR Slot ← MIN(J.Deadline, MaximumDeadline)
                DOWNTO 1 DO
         IF Schedule[Slot] = EMPTY THEN
            Schedule[Slot] ← J.ID
            TotalProfit ← TotalProfit + J.Profit
            SlotFound ← TRUE
            BREAK
         END IF
      END FOR
   END FOR
   RETURN Schedule, TotalProfit
END
```

Task 2.4 – Desk-Check (1 Mark)

Job	Deadline	Profit
A	2	80
B	1	35
C	2	60
D	3	40
E	1	25

Maximum deadline = 3. Therefore, three slots are available. Jobs sorted by descending profit: A, C, D, B, E.

Job	Slot Checked	Decision	Schedule Update
A	2	Accepted	Slot 2 = A
C	2, 1	Accepted	Slot 1 = C
D	3	Accepted	Slot 3 = D
B	1	Rejected	Slot occupied
E	1	Rejected	Slot occupied

Slot 1	Slot 2	Slot 3
C	A	D

Total profit = 60 + 80 + 40 = 180.

Task 2.5 – Challenge Your Design (1 Mark)

Modified input: Change the profit of job D from 40 to 90.

Prediction: The schedule order will change because job D now has the highest profit and will be considered first.

Job	Deadline	Modified Profit
D	3	90
A	2	80
C	2	60

B	1	35
E	1	25

Relevant decisions: D occupies slot 3; A occupies slot 2; C occupies slot 1; B and E cannot be scheduled.

Slot 1	**Slot 2**	**Slot 3**
C	A	D

Total profit = 60 + 80 + 90 = 230. The prediction about the changed job priority and profit was correct. The slot arrangement remains the same, but the selection order and total profit change.

TASK 3 – FROM RECURRENCE TO EXECUTABLE LOGIC

Dynamic Programming – 0/1 Knapsack (9 Marks)

Task 3.1 – Define the State (1 Mark)

Let DP[i][w] represent the maximum value obtainable by considering the first i items with a maximum capacity of w. Here, i represents the number of items considered, w represents the available capacity, and DP[i][w] represents the maximum achievable value.

Task 3.2 – Develop the Recurrence (2 Marks)

Case 1 – Item cannot be included: If the weight of the current item exceeds the available capacity, the item is rejected.

DP[i][w] = DP[i−1][w]

Case 2 – Item can be included: Compare excluding the current item with including it.

DP[i][w] = MAX(DP[i−1][w], Value[i] + DP[i−1][w−Weight[i]])

Boundary conditions: DP[0][w] = 0 and DP[i][0] = 0. If there are no items or capacity is zero, the maximum value is zero.

Task 3.3 – Translate the Recurrence into Pseudocode (4 Marks)

ALGORITHM ZeroOneKnapsack(Items, Capacity)
INPUT:
Items containing ID, Weight and Value

```
    Capacity – maximum carrying capacity
OUTPUT:
    Maximum Value and Selected Items
BEGIN
    n ← Number of items
    CREATE DP[0 ... n][0 ... Capacity]
    FOR i ← 0 TO n DO
        DP[i][0] ← 0
    END FOR
    FOR w ← 0 TO Capacity DO
        DP[0][w] ← 0
    END FOR
    FOR i ← 1 TO n DO
        FOR w ← 1 TO Capacity DO
            IF Items[i].Weight > w THEN
                DP[i][w] ← DP[i-1][w]
            ELSE
                Include ← Items[i].Value +
                        DP[i-1][w-Items[i].Weight]
                Exclude ← DP[i-1][w]
                DP[i][w] ← MAX(Include, Exclude)
            END IF
        END FOR
    END FOR
    MaximumValue ← DP[n][Capacity]
    CREATE SelectedItems as EMPTY
    w ← Capacity
    FOR i ← n DOWNTO 1 DO
        IF DP[i][w] ≠ DP[i-1][w] THEN
            ADD Items[i].ID to SelectedItems
            w ← w - Items[i].Weight
        END IF
    END FOR
```

```
    RETURN MaximumValue, SelectedItems
END
```

Task 3.4 – Desk-Check (1 Mark)

Item	Weight	Value
P	2	12
Q	3	20
R	4	25
S	5	30

Vehicle capacity = 7.

Items Considered	Capacity	Maximum Value
P	2	12
P, Q	5	32
P, Q	7	32
P, Q, R	7	37
P, Q, R, S	7	45

The best feasible combination is Q + R: total weight = 3 + 4 = 7 and total value = 20 + 25 = 45. Therefore, the selected items are Q and R, and the maximum value is 45.

Task 3.5 – Explain the Dependency (1 Mark)

Dynamic programming states depend on previously calculated smaller states. Each state DP[i][w] uses values from the previous item row DP[i−1]. Therefore, the previous row must be completed before calculating the current row. Processing items from 1 to n ensures that all required dependency states are already available.

TASK 4 – PSEUDOCODE FORENSICS

Find, Explain and Repair an Algorithmic Defect (7 Marks)

Task 4.1 – Locate the Defect (2 Marks)

The defective decision checks only whether the substring belongs to the dictionary. It does not check whether the preceding part of the text can also be segmented. The correct decision must check both that the previous prefix is segmentable and that the current substring belongs to the dictionary.

```
IF substring(Text, j, i) is in Dictionary THEN
    DP[i] ← TRUE
```

Task 4.2 – Construct a Failing Input (2 Marks)

Text: apple

Dictionary: {app, le, ple}

Expected result: FALSE. The complete text cannot be segmented using these words.

i	Substring Checked	Dictionary Match	Faulty DP[i]
1	a	No	FALSE
2	ap	No	FALSE
3	app	Yes	TRUE
4	ppl / pp	No	FALSE
5	ple	Yes	TRUE

At i = 5, the algorithm finds “ple” in the dictionary and sets DP[5] to TRUE, even though the prefix before it is not segmentable. The first incorrect state is DP[5] = TRUE; the expected answer is FALSE.

Task 4.3 – Repair the Algorithm (2 Marks)

Only the required condition is modified by adding DP[j] = TRUE.

```
ALGORITHM WordBreak(Text, Dictionary)
INPUT:
    Text – string of length n
    Dictionary – collection of valid words
OUTPUT:
    TRUE if Text can be segmented
```

```
    FALSE otherwise
BEGIN
    DP[0] ← TRUE
    FOR i ← 1 TO n DO
        DP[i] ← FALSE
        FOR j ← 0 TO i-1 DO
            IF DP[j] = TRUE AND
                substring(Text, j, i) is in Dictionary THEN
                DP[i] ← TRUE
                BREAK
            END IF
        END FOR
    END FOR
    RETURN DP[n]
END
```

Task 4.4 – Regression Check (1 Mark)

Text: apple. Dictionary: {app, le}.

Position	Substring	Previous DP State	Result
3	app	DP[0] = TRUE	DP[3] = TRUE
5	le	DP[3] = TRUE	DP[5] = TRUE

Final result: DP[5] = TRUE. The corrected algorithm rejects invalid segmentations and preserves correct results for valid segmentations.

TASK 5 – SAME GRAPH, DIFFERENT ALGORITHMIC REASONING

Dijkstra, Bellman-Ford and Floyd (7 Marks)

Edge	Weight
A → B	4
A → C	7
B → C	2
B → D	6

C → D	1
D → E	3
C → E	8

Task 5.1 – Greedy Shortest-Path Design (3 Marks)

Dijkstra's algorithm is used because all edge weights are non-negative.

```
ALGORITHM Dijkstra(Graph, Source)
INPUT:
   Directed weighted graph
   Source vertex A
OUTPUT:
   Shortest distances and predecessors
BEGIN
   FOR each vertex V DO
      Distance[V] ← ∞
      Predecessor[V] ← NULL
      Visited[V] ← FALSE
   END FOR
   Distance[Source] ← 0
   WHILE an unvisited vertex with finite distance exists DO
      U ← Unvisited vertex with minimum Distance
      Visited[U] ← TRUE
      FOR each neighbour V of U DO
         IF Visited[V] = FALSE THEN
            NewDistance ← Distance[U] + Weight(U,V)
            IF NewDistance < Distance[V] THEN
               Distance[V] ← NewDistance
               Predecessor[V] ← U
            END IF
         END IF
      END FOR
   END WHILE
```

RETURN Distance, Predecessor

END

Selected Vertex	**A**	**B**	**C**	**D**	**E**
Initial	0	∞	∞	∞	∞
A	0	4	7	∞	∞
B	0	4	6	10	∞
C	0	4	6	7	14
D	0	4	6	7	10
E	0	4	6	7	10

Vertex	**Shortest Distance**	**Shortest Path**
A	0	A
B	4	A → B
C	6	A → B → C
D	7	A → B → C → D
E	10	A → B → C → D → E

Task 5.2 – Change the Graph Assumption (2 Marks)

Dijkstra's algorithm assumes that all edge weights are non-negative. If negative-weight edges are introduced, permanently selecting the currently minimum-distance unvisited vertex can become unsafe because a later path may reduce its distance.

```
ALGORITHM BellmanFord(Graph, Source)
BEGIN
  FOR each vertex V DO
    Distance[V] ← ∞
  END FOR
  Distance[Source] ← 0
  FOR i ← 1 TO NumberOfVertices - 1 DO
    FOR each edge (U,V) with weight W DO
      IF Distance[U] ≠ ∞ AND
```

```
            Distance[U] + W < Distance[V] THEN
              Distance[V] ← Distance[U] + W
          END IF
        END FOR
      END FOR
      FOR each edge (U,V) with weight W DO
        IF Distance[U] ≠ ∞ AND
          Distance[U] + W < Distance[V] THEN
          RETURN "Reachable Negative Cycle Exists"
        END IF
      END FOR
      RETURN Distance
    END
```

Bellman-Ford repeatedly relaxes all edges for V−1 iterations. It does not permanently commit to a vertex's distance after one selection. A further possible improvement after these iterations indicates a reachable negative-weight cycle.

Task 5.3 – Change the Requirement (2 Marks)

Floyd state definition: DP[i][j][k] represents the shortest distance from vertex i to vertex j, using only the first k vertices as intermediate vertices.

State transition:

DP[i][j][k] = MIN(DP[i][j][k−1], DP[i][k][k−1] + DP[k][j][k−1])

```
    ALGORITHM FloydWarshall(Graph)
    BEGIN
      INITIALISE Distance[i][j] as follows:
        IF i = j THEN
          Distance[i][j] ← 0
        ELSE IF edge i → j exists THEN
          Distance[i][j] ← Weight(i,j)
        ELSE
          Distance[i][j] ← ∞
        END IF
      FOR k ← 1 TO NumberOfVertices DO
```

```
        FOR i ← 1 TO NumberOfVertices DO
            FOR j ← 1 TO NumberOfVertices DO
                Distance[i][j] ← MIN(
                    Distance[i][j],
                    Distance[i][k] + Distance[k][j]
                )
            END FOR
        END FOR
    END FOR
    RETURN Distance
END
```

Difference: Dijkstra and Bellman-Ford calculate shortest paths from one source vertex, whereas Floyd-Warshall calculates shortest paths between every pair of vertices.

TASK 6 – DESIGN CHALLENGE

Energy-Aware Job Scheduling in a Cloud Data Centre (13 Marks)

Stage 1 – Interpret the Problem (2 Marks)

The cloud data centre needs to select jobs that do not overlap and provide the maximum total energy-saving value.

Component	**Description**
Input	Job ID, start time, finish time and energy-saving value.
Output	Selected non-overlapping jobs and maximum total energy-saving value.
Objective	Maximise the sum of energy-saving values.
Constraints	Only one job can execute at a time. Overlapping jobs cannot both be selected.
Decision	Whether to include or exclude each job from the schedule.

This is an optimisation problem involving compatible job intervals.

Stage 2 – Propose the Strategy (2 Marks)

Strategy	Limitation
Earliest finishing time	May select jobs with low energy-saving values.
Highest energy-saving value	May select one long job and miss several valuable compatible jobs.
Shortest duration	May ignore jobs with higher energy-saving values.
Dynamic Programming	Considers combinations of compatible jobs and compares their total values.

Selected strategy: Dynamic Programming – Weighted Interval Scheduling.

Justification: A locally attractive job may prevent the selection of multiple other jobs. Selecting jobs only by their individual values or finishing times does not guarantee the maximum total energy-saving value. Dynamic programming considers both possibilities—include and exclude—and compares their total values. This provides an optimal solution for the general problem.

Stage 3 – Identify the Critical Information (2 Marks)

State definition: DP[i] represents the maximum total energy-saving value obtainable using the first i jobs after sorting them by increasing finish time.

- Job start time.
- Job finish time.
- Energy-saving value.
- Previous compatible job.
- Maximum value of previously considered jobs.

Let P(i) represent the index of the last job that finishes on or before the start time of job i.

Recurrence: DP[i] = MAX(DP[i−1], EnergyValue[i] + DP[P(i)]).

Base condition: DP[0] = 0. If no previous job is compatible, P(i) = 0.

Stage 4 – Design Complete Pseudocode (4 Marks)

```
ALGORITHM EnergyAwareScheduling(Jobs)
INPUT:
    Jobs containing:
```

```
    Job ID
    Start Time
    Finish Time
    Energy-Saving Value
OUTPUT:
  Selected Jobs and Maximum Energy-Saving Value
BEGIN
  n ← Number of jobs
  SORT Jobs in ascending order of Finish Time
  CREATE DP[0 ... n]
  CREATE Previous[1 ... n]
  DP[0] ← 0
  FOR i ← 1 TO n DO
    Previous[i] ← 0
    FOR j ← i-1 DOWNTO 1 DO
      IF Jobs[j].FinishTime ≤ Jobs[i].StartTime THEN
        Previous[i] ← j
        BREAK
      END IF
    END FOR
  END FOR
  FOR i ← 1 TO n DO
    IncludeValue ← Jobs[i].EnergyValue +
                DP[Previous[i]]
    ExcludeValue ← DP[i-1]
    IF IncludeValue > ExcludeValue THEN
      DP[i] ← IncludeValue
    ELSE
      DP[i] ← ExcludeValue
    END IF
  END FOR
  CREATE SelectedJobs as EMPTY
  i ← n
```

```
    WHILE i > 0 DO
      IncludeValue ← Jobs[i].EnergyValue +
                DP[Previous[i]]
      IF IncludeValue > DP[i-1] THEN
        ADD Jobs[i].ID to SelectedJobs
        i ← Previous[i]
      ELSE
        i ← i - 1
      END IF
    END WHILE
    REVERSE SelectedJobs
    MaximumValue ← DP[n]
    RETURN SelectedJobs, MaximumValue
  END
```

Stage 5 – Desk-Check (1 Mark)

Job	**Start Time**	**Finish Time**	**Energy Value**
J1	1	3	20
J2	2	5	50
J3	4	6	30
J4	6	8	40
J5	5	9	60
J6	8	10	30
J7	9	11	50
J8	11	13	40

The jobs are already sorted by finishing time.

Job	**Previous Compatible Job**	**P(i)**
J1	None	0
J2	None	0
J3	J1	1
J4	J3	3

J5	J2	2
J6	J4	4
J7	J5	5
J8	J7	7

Job	**Include Value**	**Exclude Value**	**DP[i]**
J1	20	0	20
J2	50	20	50
J3	50	50	50
J4	90	50	90
J5	110	90	110
J6	120	110	120
J7	160	120	160
J8	200	160	200

Reconstruction from J8 selects J8, J7, J5 and J2. In chronological order: J2 → J5 → J7 → J8.

Compatibility check: J2 finishes at 5; J5 starts at 5 and finishes at 9; J7 starts at 9 and finishes at 11; J8 starts at 11 and finishes at 13. The jobs are compatible because a job may start exactly when the previous job finishes.

Total energy-saving value = 50 + 60 + 50 + 40 = 200.

Stage 6 – Attack Your Own Design (2 Marks)

Adversarial test dataset:

Job	**Start Time**	**Finish Time**	**Energy Value**
A	1	10	100
B	1	3	40
C	3	5	40
D	5	7	40
E	7	9	40

Why challenging: Job A has the highest individual energy-saving value of 100. However, selecting A prevents the selection of jobs B, C, D and E. The combined value of the four smaller jobs is greater than the value of A.

Prediction: A strategy selecting the highest-value job first will produce a poor result. The dynamic programming strategy should identify the combination of smaller compatible jobs.

Job	**Include Value**	**Exclude Value**	**DP[i]**
A	100	0	100
B	40	100	100
C	80	100	100
D	140	100	140
E	180	140	180

The optimal selected jobs are B → C → D → E. Total value = 40 + 40 + 40 + 40 = 160.

Selection	**Total Energy-Saving Value**
Job A alone	100
B + C + D + E	160

Conclusion: The adversarial input demonstrates that selecting the job with the highest individual value does not always produce an optimal schedule. Dynamic programming considers combinations of compatible jobs and selects the combination with the maximum total energy-saving value. The include-or-exclude decision is critical, and the proposed algorithm remains optimal for this test case.